Team Assignment 4 Question 3 - Architectural Patterns & Semester Project Implementation
Admissions Alchemists
Joseph Rissman, Yue Zhang, Nitin Krishna Bojji
Feb 19, 2025

Here, we compare three different architecture patterns: Model-View-Controller (MVC) architecture, Microservices architecture, and Event-Driven architecture.

First, the Model-View-Controller (MVC) divides an application into three components: Model (data and business logic), View (user interface), and Controller (handles user input). This structure allows for easier development and parallel work across teams. However, as applications grow, tight coupling can complicate updates and debugging. MVC is commonly used in web applications like content management systems and e-commerce sites.

In contrast, Microservices Architecture segments an application into independent services, each responsible for a specific function. This aids scalability and resilience, enabling separate development and deployment. However, managing multiple services can increase operational complexity and requires strong DevOps practices. Microservices are ideal for large-scale applications like SaaS platforms and streaming services.

Meanwhile, Event-Driven Architecture allows real-time communication between components through events. It's best for asynchronous processing in IoT systems and real-time analytics. While it enhances scalability, debugging can be difficult due to its asynchronous nature, and poor event handling can lead to message loss.

In summary, MVC is suitable for structured web applications, Microservices excel in distributed systems, and Event-Driven architecture is optimal for real-time processing. The best choice depends on project needs for scalability and complexity management.

For GradAid, a web application designed to support graduate school applications with AI-powered tools, the MVC pattern is the most suitable choice. While Microservices offer excellent scalability and resilience, they introduce significant operational overhead that is unnecessary for GradAid's current scope. Event-Driven Architecture, while suitable for real-time systems, adds complexity that isn't essential for GradAid's core functionalities. The MVC pattern offers a balanced approach, providing the necessary scalability and maintainability without the added complexity of a fully distributed system. The clear separation of concerns in MVC makes it easier to understand, modify, and update individual components of GradAid without affecting the entire system, which is crucial for development. Furthermore, the layered structure of MVC allows for the implementation of security measures at different levels, enhancing the overall security of the application.

A key feature in GradAid is the AI-powered SOP/LOR generator, and the MVC architecture directly supports its implementation. The Model layer, comprising FastAPI and Supabase/PostgreSQL, handles data persistence, including user information, application details, and generated SOP/LOR content. It also interacts with the LangChain AI engine to process prompts and retrieve generated text. The View layer, built with Next.js, provides the

user interface for interacting with the SOP/LOR generator, allowing users to input information, customize prompts, and view the generated documents in real time. The Controller layer, managed by FastAPI, acts as an intermediary between the View and Model. It receives user requests, processes them, interacts with the Model to retrieve or store data, and sends the results back to the View. This layer also manages communication with the LangChain AI engine, orchestrating the generation process. The MVC architecture enables a clean and structured implementation of the AI-powered SOP/LOR generator. The separation of concerns ensures that each component focuses on its specific task, promoting code reusability, maintainability, and testability. This flexibility is crucial for incorporating future enhancements and improvements to the AI generation capabilities. The structured data flow between the layers also ensures efficient processing and a smooth user experience.